A Unified Framework for Automating Software Security Analysis in DevSecOps

Mohammad A. Aljohani
College of Computer and Information Science Department of Computer Science
AL Imam Mohammad bin Saud Islamic University
Riyadh, Saudi Arabia
moaaljohani@sm.imamu.edu.sa

Sultan S. Alqahtani
College of Computer and Information Science Department of Computer Science
AL Imam Mohammad bin Saud Islamic University
Riyadh, Saudi Arabia
ssalqahtani@imamu.edu.sa

2023 International Conference on Smart Computing and Application (ICSCA) | 979-8-3503-4705-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICSCA57840.2023.10087568

***Abstract*— The Development and Operations (DevOps) methodology is a set of practices and cultural values. Its main objectives are to shorten the software development lifecycle, produce quality software, and eliminate software evolution barriers. The increased demand for secure software applications has led to a new field version of DevOps called Development, Security, and Operations (DevSecOps), which attempts to integrate security practices into the DevOps process. In this paper, we outline the current challenges in securing DevOps applications, such as the lack of automated software security testing tools, insufficient integration of security tools, a lack of security knowledge between developers, and false-positive results produced by many vulnerability scanner tools. Therefore, we introduce a unified framework for automating software security analysis in the DevSecOps paradigm that serves as a middle development process between software applications' Continuous Integration (CI) and Continuous Delivery (CD) pipelines and application security services. We have shown the framework's high-level architecture, and one case study is presented to illustrate the applicability of our proposed approach.**

Keywords*— *DevSecOps, Security, Automation, DevOps, DAST, SAST

I. Introduction

Software development techniques are being changed to improve collaboration and project consistency between developers and operations teams [1]. In the software industry, "*the quicker you can release new features and fix bugs, the faster you can respond to your customers' needs and build a competitive advantage*" [2]. To achieve that, the software development paradigms have evolved into new approaches that integrate the software development and IT operation environments. This new era of software development environment is called DevOps [3].

DevOps is a set of practices and cultural values whose main objective is to shorten the systems development lifecycle and eliminate software evolution barriers to improve the collaboration and communication between the development and IT operation teams. Additionally, it provides Continuous Integration (CI) to support software quality and accelerate product releases [4]. DevOps focuses on rapid software development and delivery via the Agile methodology to increase collaboration and decrease team inconsistencies. However, the agile methods focused on shorter duration iterations to produce working applications for end-users and get immediate feedback [5]. On the other hand, security requirements are frequently overlooked in running applications to facilitate velocity in business agility. Security is commonly considered the last step to be checked when an application has already been developed and is ready for release. Shifting security to the left and engaging security professionals from the beginning of the development process makes it easier to plan and implement the integration of security controls throughout the development process without incurring any delays [6]. And it will decrease the software security vulnerabilities before getting known to the public. However, the increased demand for secure software applications has led to a new field version of DevOps called DevSecOps, which tries to integrate Security (Sec) practices into the DevOps paradigm [6]. DevSecOps integrates security practices and generates security and compliance artifacts into the pipeline to ensure that security resolutions are delivered as part of all DevOps practices [7].

This concept is an effort to develop and combine new security procedures that can be used in the fast-paced and agile world of DevOps. It extends DevOps' goal of encouraging collaboration between developers and operators by engaging security specialists from the beginning of the software development lifecycle [4]. However, integrating security practices into the DevOps model creates several challenges. Recent studies [8] and [9] showed that the most critical challenge in securing DevOps implementations is the lack of empowering automated security testing tools. The Security Compass[1] conducted a survey [10] in the software industry and found that 51% of the respondents highlighted the following concern: "*insufficient automation of security testing is the most common security challenge in the current DevSecOps implementation*.". Another recent survey conducted by Checkmarx[2] [11] stated that 31% of the survey participants do not have application security testing integration within the development tools; as a result, the lack of security automation. Furthermore, because of the lack of integration of security tools, 62% of participants specified that the improved integration of the security tools was the top priority of their needs. Insufficient integration of security tools precludes visibility and prevents teams from achieving the level of automation needed to improve release velocity and product quality. Also, the need for knowledge of when and where to integrate security solutions into DevOps practices is a significant problem for software practitioners and prevents them from automating security processes [12].

II. Problem Statement

DevOps focuses on shorter development cycles known as *increments* or *iterations*, which increase the speed of deployment frequency, and more reliable releases closely aligned with business goals. Simultaneously, applications are expected to be secure against threats and known vulnerabilities. In traditional software development paradigms, security tests are performed in the "Testing Environment" after the code development is completed or

[1] https://www.securitycompass.com

[2] https://checkmarx.com

later in the "Acceptance Environment." After passing the tests, tagged as "secure" and distributed to customers or deployed to the production environment. Currently, organizations are trying to apply the DevSecOps model by adding security tools in the CI/CD pipelines to improve security and automate security analysis with the workflow of the software life cycle.

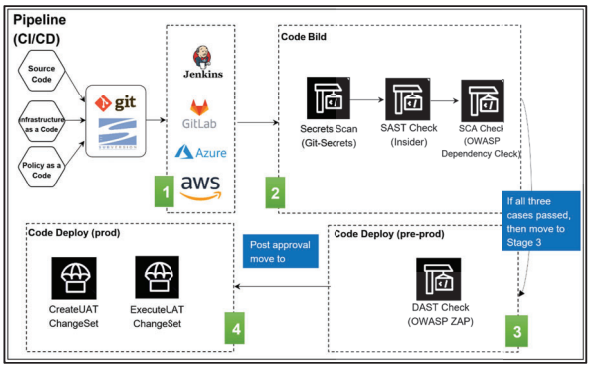

Fig. 1. Traditional CI/CD pipeline

Different types of security tests are available, such as Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST), Infrastructure as Code (IaC), and Software Composition Analysis (SCA). Commercial software security products usually support these testing techniques. Enterprises commonly purchase various security tools and integrate them manually into the CI pipelines of in-house apps to improve application security posture. The traditional integration model is demonstrated in Fig. 1.

The Traditional integration model presents many challenges, as shown in red color numbers in Fig. 2, and we explain them in detail as follows:

(1) Scanning with several security tools: each application security tool introduces a new step into the CI/CD pipeline. As a result, it will extend the CI execution time and increase the complexity of configuring these processes. More than adding security tools into the CI/CD pipelines during the development phase is required; it can become useless if not configured correctly. Many companies and businesses are still determining whether their security tools are working correctly; for example, AttackIQ shows that 53% of IT companies need to know if their security tools work or not [13].

(2) Taking a long time during the pipeline for security testing: some security scans need extensive scanning time, impacting the CI/CD pipeline's overall execution duration. For example, the DAST scanning agent may take more than two hours to scan one web application, while the project's current CI/CD execution duration is less than 10 minutes [14].

(3) False-positive results: a high number of false positives delays product releases, and developers eventually lose trust in the security tool's ability. Hence, results need to be enhanced [15].

(4) Lack of knowledge: software developers may need to learn when and where to integrate and utilize the existing security tools and add them to the pipeline; due to this, they require assistance from the cyber security team [16].

(5) Security regulatory requirements: because each application is handled separately, keeping track of the security service status and meeting the regulatory requirements is challenging. Also, accurate data is required to help when needed for external security services and make a better decision for management to determine which security services are worth the cost.

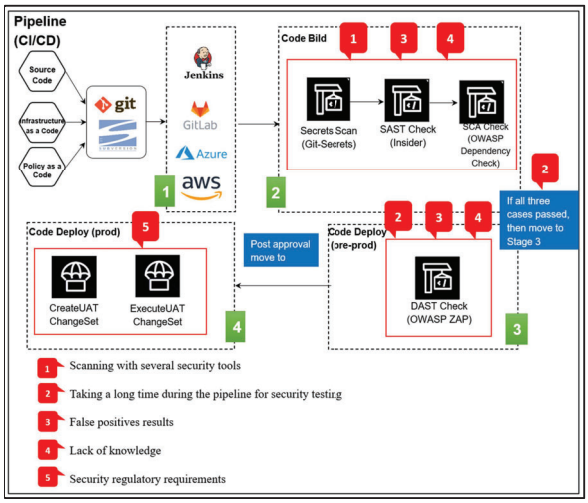

Fig. 2. Mapped challenges to the traditional integration model

Given the growing importance of information security and the challenges we mentioned above to integrate security practices implementations automatically in traditional DevOps paradigms, we proposed a framework approach to address the DevOps software security applications' challenges. The following section will detail the framework architecture and explain its essential components.

III. Related Work

Embedding security services within the CI/CD pipeline and fully automated setup in a production environment is valuable for finding bugs and vulnerabilities quickly, especially in an inexperienced team in cybersecurity.

Rangnau et al. [14] studied and explained how to conduct Dynamic Application Security Testing (DAST) in a CI/CD pipeline. They looked at the difficulties of integrating DAST. They were also convinced that there needs to be more literature focusing on DAST and describing how to integrate security tools to scan for vulnerabilities in a pipeline. They performed a case study by integrating three commonly known security testing tools into a CI/CD pipeline.

Another research contribution conducted by Kumar and Goyal [17] which they created a conceptual model for automated DevSecOps using open-source software with several key metrics. The introduced metrics measure the velocity of a successful CI/CD pipeline, as they mapped the security controls to the practices and workflow stages in the Automated DevSecOps using the Open-source software over Cloud (ADOC) model. However, the model lacks generalization, which cannot be used in two different clouds.

Rafi et al. [8] extracted the security challenges related to the DevOps approach from the literature and evaluated them using a questionnaire survey. They concluded that the lack of automated testing tools is the most critical challenge to secure DevOps implementations. As a result, they advised that an appropriate automation test should be to monitor the security risks of DevOps.

Akbar et al. [18] performed a questionnaire-based survey to identify and prioritize challenges associated with implementing DevSecOps. They identified 18 challenges for DevSecOps, and they concluded that the lack of secure coding standards and automated testing tools for security is the most critical challenge in DevOps implementations.

The review studies [9] and [15] identified three significant challenges in the current DevSecOps methods, where most of them share the same common issues:

- Lack of DevSecOps security experts,
- Lack of the choice of security tools, the complexity of integrating security tools, and the configuration management of security tools.
- Lack of mature DevSecOps solutions

Unlike most related works, the **Unified Framework for Automating Software Security Analysis in DevSecOps** will enable DevSecOps operators to fully automate the integration of the necessary security tools and procedures with customized configuration based on best security practices. In contrast to the previous studies, the framework also supports getting immediate feedback into a CI/CD pipeline at runtime. Moreover, improving the results of security tools by refining the results, detecting the correct vulnerabilities, and minimizing false positives is one of the objectives of this framework, which related studies have yet to explore.

IV. A Unified Framework for Automating Software Security Analysis in the DevSecOps

This paper aims to show our proposed framework's design and development process. Its main goal is to serve as middleware between software applications CI/CD pipelines and application security services that must be called during the pipeline execution and return results asynchronously. We can highlight the difficulties inherent in the practical workload required to achieve this integration. We show the magnitude of these problems and the elementary ways to solve them using an actual case study (see section IV).

A. Framework Architecture

Fig. 3. depicts the high-level architecture of the framework. The framework is divided into two main parts: the **agent** and the **engine**. The **agent** is integrated as a step in a project's CI pipeline to collect project details (project name, source code, etc.) and push this information to the engine. The **engine** handles the received data from the agent and performs the primary tasks (such as security vulnerability scanning, etc.). The framework's **engine** will be built with a microservice architecture which can give the following advantages:

- Modules can be deployed independently: The engine, as shown in the Fig. 3, supports various security activities (e.g., vulnerability management, service (security scanning), and reporting). Furthermore, each feature is defined as an independent microservice that can be designed and developed separately by the microservice architecture. Implementing a new feature as a microservice does not require restarting the entire engine.
- Highly maintainable and available: When one module fails, it does not affect the other components.
- Scalability: Each service may be scaled independently to offer more resources as required.

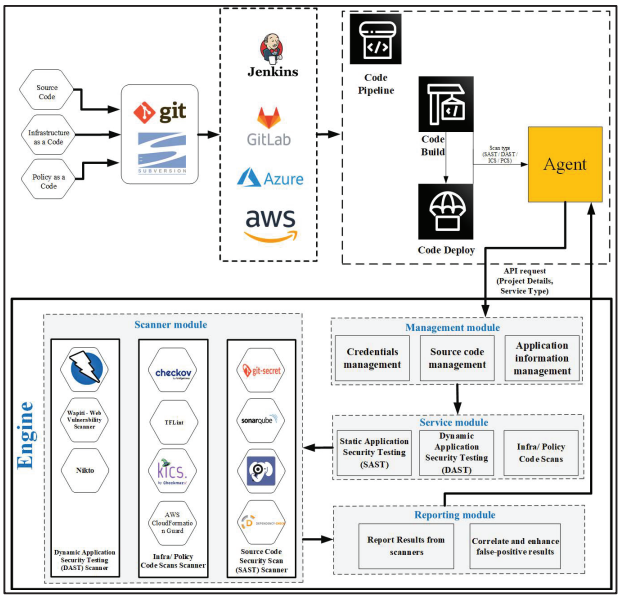

Fig. 3. Framework high-level architecture

As shown in the Fig. 3, each module can be defined as follow:

The **agent**: is an API integrated as a step in a CI pipeline and is responsible for transferring project details, as we mentioned earlier, from CI/CD to the **engine's** management module.

The **engine**: is responsible for receiving the agent's data and performing the required actions in a fully automatic process. And it is divided into four modules:

- The management module: it is in charge of interacting with the agent. When the CI pipeline (via REST API) requests a new security test with a given service type scope (SAST or DAST), the management module uses the defined functions (managing application information, managing credentials, and managing source code) to generate a new ID for the project and store project details within the framework cash database.
- The service module: this module interacts with the management module to get the service type scope (SAST/DAST/IaC) and launch the specified and related security tools in the scanner module.
- The scanner module: it is responsible for all security tools and how each tool is implemented and configured. It provides a variety of scanning tools and helps users to choose the intended scanning tool based on the selected service. Additionally, it is designed to seamlessly integrate new tools based on users' needs.
- The reporting module: allows DevSecOps operators to see the security tools' results in just in time with valuable information and provides a user-friendly web interface. Moreover, it is responsible for storing the vulnerabilities from multiple types of security tools scanners in one place. In addition, the reporting module provides a flexible and extensible API that

DevSecOps operators can develop to build their reports.

V. Case Study

This section introduces the case study we use to evaluate the applicability of our framework approach. More specifically, we identify one vulnerable software application (Damn Vulnerable Java Application (DVJA) [19]) that exists in the GitHub repository and contains security vulnerabilities disclosed publicly for testing purposes. This case study aims to determine how the main components of the framework are used and test the applicability of the different modules. Additionally, evaluate the automation feature between the agent and other modules.

In detail, we have selected a Security Orchestration, Automation, and Response (SOAR) technology for the implementation. The SOAR technology enables the coordination, execution, and automation of tasks between tools, all within a single platform. The simulation process has been done with the help of a popular tool called Shuffle [20]. Shuffle is an open-source interpretation of SOAR. With plug-and-play apps, it seeks to bring all the features required to implement the framework, transmit data throughout a workflow, and make automation approachable for the whole process. Workflows are the component of Shuffle that bring everything together. Shuffle gives the framework more strength by enabling it to deploy new, complex (or essential) tools in minutes instead of hours or days. Additionally, The whole framework is implemented using Docker [21]. The Docker containers are deployed and managed in this implementation by Docker Compose. Then the framework is deployed and tested in a local virtual machine that simulates a real-life environment. Table I and Table II describe the virtual machine specifications and the Shuffle environment, respectively.

Table I. Server hardware and software specifications

Item	Specification
Processor	Intel Core i7 – 4
Memory	9 GB
Hard Disk	50 GB – SSD
Operating System	Ubuntu 22.04 LTS

Table II. Shuffle environment

Component type	Technology used
Frontend	ReactJS [22]
Database	Opensearch [23]
Backend	Golang [24]
Orborus	
Worker	
app SDK	Python [25]

We have implemented the main parts: the agent and some engine modules. However, by doing so, our framework takes advantage of using Shuffle to achieve the following:

- Automate any tool by creating a new custom application (e.g., using Python [25] scripts) and integrating it with the other instruments.
- All components and services in the framework are built using Docker containers, even in case of compromise; this keeps the environment safe. It allows us to sandbox and control how actions are performed.
- Each service can be scaled separately to provide extra resources if needed.

After running the Shuffle tool using a docker-compose, we started building the main parts. The first part is the **agent** (see Fig. 3). We have created a custom app using Python, which receives the following: Name (variable name of type String), Scan_Service_type (to identify whether the service the scanning type is SAST or DAST), and URL (the path of the source code repository such as Git repository).

The components of the **agent** application contain: `api.yaml`, which includes application configuration; `dockerfile` for build instructions, `readme.md` to document the application, `requirements.txt` which consists of any additional packages; and `app.py` for python script, which includes the necessary functions needed for implementation.

After the agent application was built, we implemented the second part, which is the **engine**. As mentioned in the Fig. 3, the **engine** contains multiple modules. In this case study, we tested only two modules, management and scanner modules.

First, the management module: all the information (e.g., project name, URL, Scan_Service_type) from the agent is stored in persistent storage within Shuffle.

Second, the scanner module: we have created two types of security scanners in this module, two tools for SAST, and one tool for DAST, as follows:

- SAST tools include: Dependency-Check, which attempts to detect publicly disclosed vulnerabilities within a project's dependencies [26]. And Detect Secrets tool, which is built on Checkov, a static code analysis tool for scanning infrastructure as code (IaC) files for misconfigurations that may lead to security or compliance problems [27].
- DAST tools include: Nikto Scanner, a web server scanner that performs comprehensive tests against web servers for multiple items [28].

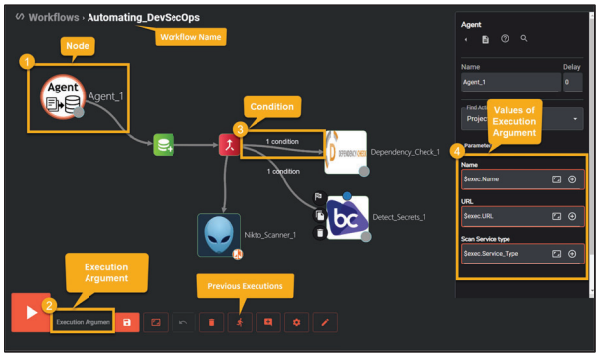

Fig. 4. Automating DevSecOps workflow.

All of these tools are integrated within a single workflow. Workflows are considered the backbone of Shuffle [20]. The main components in each workflow are illustrated in Fig.4. Each workflow uses applications, triggers, conditions, and variables to create automation quickly. We can check all previous workflows' executions and the last executions' status and results. In the workflow, nodes (Fig.4.- number 1) are objects to present the app or trigger. Conditions (Fig.4.- number 3) determine the flow of node execution inside a workflow. In our case, we run Scan Service, a type of SAST,

and in this situation, the framework will run the Dependency-Check and Detect Secrets tools. On the other hand, if the Scan Service type is DAST, the framework will run the Nikto tool. The execution argument can be a string or number (Fig.4.- numbers 2 and 4). Variables are commonly used to store API keys, URLs, or usernames. As illustrated in Figures 4 and 5, we have created a new workflow in the Shuffle, added all the tools, and integrated the workflow with the CI/CD. Once we trigger the workflow from any CI/CD by calling the API, it will be executed automatically.

For the case study application, we tested the applicability of our framework, DVJA application was used. We integrate the framework with the CI/CD and send the following HTTP request:

- URL: http://IP_Address:3001/api/v1/workflows/{workflow_id}/execute
- Header: "Authorization: Bearer API_Key"
- Body as JSON: "{'Name': 'Demo', 'URL':'https://github.com/appsecco/dvja.git','Service_Type':'SAST'}"

We conducted a first evaluation of the framework. We only need to send the project information, and all the tools will work without any intervention from the developers, unlike the manual process that needs to integrate and set up each tool independently in the beginning during the CI/CD. The Shuffle portal interface allows examining each test's outcomes and the security tools' results; for example, the Dependency-Check tool results in Fig. 6. All the security tests (DAST and SAST) have detected vulnerabilities. Fifteen minutes and fifty seconds are needed to complete all security test scenarios with the evaluation. This run-time includes performing the security tests and exporting the detected vulnerabilities to the engine. The execution of SAST takes four minutes and thirty seconds in total. The SAST test identified four vulnerabilities, and according to severity, there was one "Low", three "Medium", and zero "High" risk. The execution of DAST takes fifty seconds and found twenty known vulnerabilities. However, as the Nikto tool lacks a severity feature [29], it is up to the consultants to assign a severity to the vulnerabilities. For the time being, our framework will consider this as future work.

Finally, the evaluation of SAST and DAST tests took ten minutes and thirty seconds in total to read the results from the portal. An overview of these results can be found in Fig. 7.

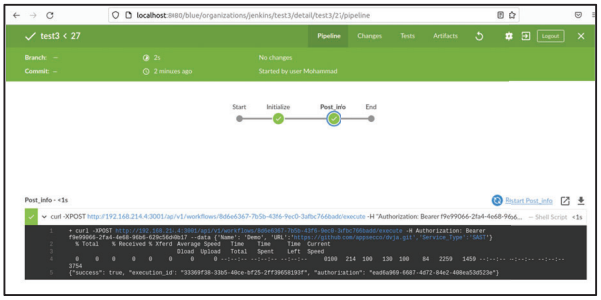

Fig. 5. Jenkins pipeline

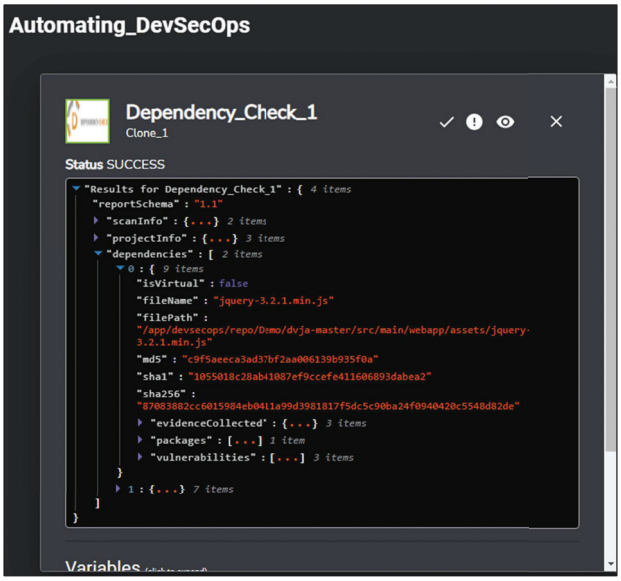

Fig. 6. Dependency-Check tool results

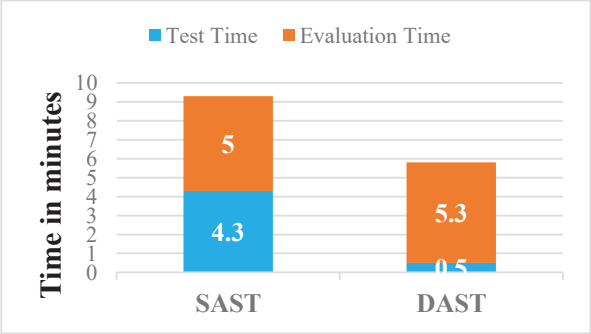

Fig. 7. Test and evaluation time for all pipeline stages

VI. Conclusion and Future Work

This paper outlines the security challenges and concerns facing the current DevOps environment. Based on different parameters covered in the literature review studies, experience reports, concept papers, surveys, etc., we proposed a unified framework for automating software security analysis in DevSecOps that serves as a middle process between software application CI/CD pipelines and application security services. The security requirements identified through the DevSecOps model challenges analysis are addressed through this framework. The framework will enable software product producers to deliver secure applications and services to the market with accelerated speed and sustainable agility. The paper illustrates the framework's high-level design, implementation, and workflow between the components during the pipeline.

Potential future work includes extending the functionalities of the framework by adding more modules not included in the current version of the framework (e.g., user-reports analysis dashboard). We evaluate the proposed framework by conducting a user study to measure the actual developer's feedback and conducting quantitative and qualitative experimental studies. Finally, reduce the false positive results introduced by the integrated security tools.

REFERENCES

[1] R. W. Macarthy and J. M. Bass, "An Empirical Taxonomy of DevOps in Practice," in *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, Aug. 2020, pp. 221–228. doi: 10.1109/SEAA51224.2020.00046.

[2] A. Davis, "DevOps," in *Mastering Salesforce DevOps*, Berkeley, CA: Apress, 2019, pp. 27–64. doi: 10.1007/978-1-4842-5473-8_3.

[3] "10+ Deploys Per Day: Dev and Ops Cooperation at Flickr - dev2ops." dev2ops.org, May 2022. Accessed: May 10, 2022. [Online]. Available: http://dev2ops.org/2009/06/10-deploys-per-day-dev-and-ops-cooperation-at-flickr/

[4] W. P. Luz, G. Pinto, and R. Bonifácio, "Adopting DevOps in the real world: A theory, a model, and a case study," *Journal of Systems and Software*, vol. 157, p. 110384, Nov. 2019, doi: 10.1016/j.jss.2019.07.083.

[5] F. Colavita, "DevOps Movement of Enterprise Agile Breakdown Silos, Create Collaboration, Increase Quality, and Application Speed," 2016, pp. 203–213. doi: 10.1007/978-3-319-27896-4_17.

[6] H. Myrbakken and R. Colomo-Palacios, "DevSecOps: A Multivocal Literature Review," 2017, pp. 17–29. doi: 10.1007/978-3-319-67383-7_2.

[7] "DevSecOps CSRC." csrc.nist.gov, Oct. 2020. Accessed: Apr. 13, 2022. [Online]. Available: https://csrc.nist.gov/Projects/devsecops

[8] S. Rafi, W. Yu, M. A. Akbar, A. Alsanad, and A. Gumaei, "Prioritization Based Taxonomy of DevOps Security Challenges Using PROMETHEE," *IEEE Access*, vol. 8, pp. 105426–105446, 2020, doi: 10.1109/ACCESS.2020.2998819.

[9] R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, "Challenges and solutions when adopting DevSecOps: A systematic review," *Inf Softw Technol*, vol. 141, p. 106700, Jan. 2022, doi: 10.1016/j.infsof.2021.106700.

[10] "Survey: The 2021 State of DevSecOps." resources.securitycompass.com, Feb. 2021. Accessed: Mar. 25, 2022. [Online]. Available: https://resources.securitycompass.com/technology/2021-state-of-devsecops

[11] "2021 State of DevSecOps Report." checkmarx.com, Nov. 2021. Accessed: Mar. 27, 2022. [Online]. Available: https://checkmarx.com/resources/reports/2021-state-of-devsecops-report

[12] N. Tomas, J. Li, and H. Huang, "An Empirical Study on Culture, Automation, Measurement, and Sharing of DevSecOps," in *2019 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*, Jun. 2019, pp. 1–8. doi: 10.1109/CyberSecPODS.2019.8884935.

[13] "Ponemon Study: 53% of IT Leaders Don’t Know if Cybersecurity is Working." go.attackiq.com, Jul. 2019. Accessed: Apr. 02, 2022. [Online]. Available: https://go.attackiq.com/PR-2019-PONEMON-REPORT_LP.html

[14] T. Rangnau, R. v. Buijtenen, F. Fransen, and F. Turkmen, "Continuous Security Testing: A Case Study on Integrating Dynamic Security Testing Tools in CI/CD Pipelines," in *2020 IEEE 24th International Enterprise Distributed Object Computing Conference (EDOC)*, Oct. 2020, pp. 145–154. doi: 10.1109/EDOC49727.2020.00026.

[15] R. Mao *et al.*, "Preliminary Findings about DevSecOps from Grey Literature," in *2020 IEEE 20th International Conference on Software Quality, Reliability and Security (QRS)*, Dec. 2020, pp. 450–457. doi: 10.1109/QRS51102.2020.00064.

[16] B. S. Rahul, P. Kharvi, and M. Manu, "Implementation of DevSecOps using Open-Source tools," *International Journal of Advance Research, Ideas and Innovations in Technology*, vol. 5, pp. 1050–1051, 2019.

[17] R. Kumar and R. Goyal, "Modeling continuous security: A conceptual model for automated DevSecOps using open-source software over cloud (ADOC)," *Comput Secur*, vol. 97, p. 101967, Oct. 2020, doi: 10.1016/j.cose.2020.101967.

[18] M. A. Akbar, K. Smolander, S. Mahmood, and A. Alsanad, "Toward successful DevSecOps in software development organizations: A decision-making framework," *Inf Softw Technol*, vol. 147, p. 106894, Jul. 2022, doi: 10.1016/j.infsof.2022.106894.

[19] "appsecco/dvja: Damn Vulnerable Java (EE) Application." https://github.com/appsecco/dvja (accessed Oct. 10, 2022).

[20] "Shuffle Automation - An Open Source SOAR solution." https://shuffler.io/ (accessed Sep. 10, 2022).

[21] "Docker: Accelerated, Containerized Application Development." May 2022. Accessed: Sep. 27, 2022. [Online]. Available: https://www.docker.com/

[22] "React – A JavaScript library for building user interfaces." Accessed: Oct. 01, 2022. [Online]. Available: https://reactjs.org/

[23] "OpenSearch," *OpenSearch*. Accessed: Oct. 04, 2022. [Online]. Available: https://opensearch.org/

[24] "The Go Programming Language." Accessed: Oct. 02, 2022. [Online]. Available: https://go.dev/

[25] "Welcome to Python.org," *Python.org*. Accessed: Sep. 15, 2022. [Online]. Available: https://www.python.org/

[26] "OWASP Dependency-Check." https://owasp.org/www-project-dependency-check (accessed Sep. 25, 2022).

[27] Bridgecrew, "Policy-as-code for everyone." https://www.checkov.io/ (accessed Sep. 30, 2022).

[28] "Nikto2 | CIRT.net." https://cirt.net/Nikto2 (accessed Oct. 18, 2022).

[29] "Add severities to checks · Issue #13 · sullo/nikto," *GitHub*. Accessed: Oct. 15, 2022. [Online]. Available: https://github.com/sullo/nikto/issues/13